isec Engenharia

# Trabalho Prático

## Programação Avançada

Relatório do Projeto Deep Sea Mining

Coimbra, Junho de 2026

# Índice

1. Identificação..... 3
2. Explicação do Jogo e Opções Tomadas.....3
3. Diagrama de Estados..... 4
4. Diagrama de Classes e Estrutura..... 4
5. Explicação das Opções na UI..... 5
6. Tabela de Tarefas..... 10

# 1. Identificação

**Turma:** P6

**Grupo:**

• Diogo Ribeiro Costa (Nº 2024143983) - a2024143983@isec.pt

• Rodrigo Cravo Pereira (Nº 2024117439) - a2024117439@isec.pt

• João Bernardo Libório Graça (Nº 2024128702) - a2024128702@isec.pt

# 2. Explicação do Jogo e Opções Tomadas

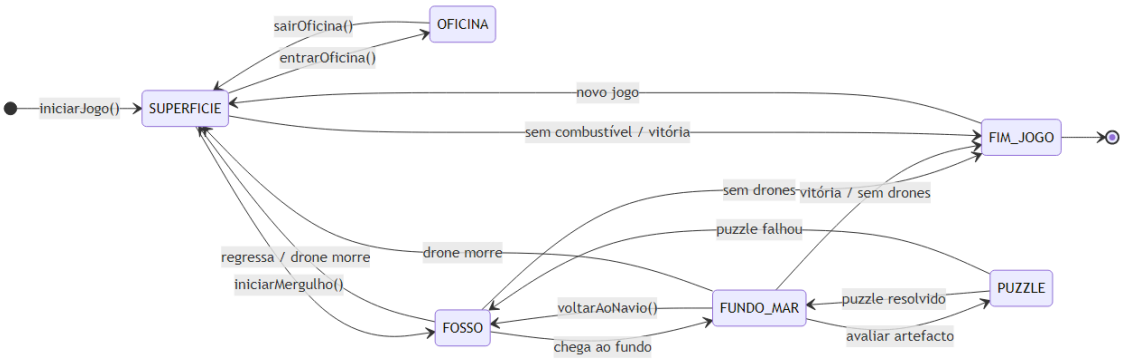
O Deep Sea Mining é um jogo de exploração e gestão de recursos onde o jogador assume o comando de um navio, e gere uma frota de drones subaquáticos com o objetivo de explorar o oceano, descer a fossos marinhos e recolher minérios e artefactos no fundo do mar, gerindo de forma crítica a integridade do casco e o combustível disponível.

Para garantir que a aplicação fosse escalável e de fácil manutenção, o grupo tomou decisões arquiteturais claras desde o início do projeto:

1. **Arquitetura MVC (Model-View-Controller):** Adotou-se uma separação estrita entre a lógica de negócio (Modelo) e a interface gráfica (View). Esta opção garantiu que o motor do jogo pudesse ser testado e executado de forma totalmente independente da tecnologia visual escolhida.
2. **Padrão Facade (DeepSeaManager):** Para proteger a interface gráfica da complexidade interna do modelo de dados, implementou-se um Facade. Esta classe atua como o único ponto de comunicação, expondo apenas os métodos estritamente necessários para a UI e centralizando a gestão dos dados e dos estados.

**Padrão State (Máquina de Estados):** A progressão do jogo (Superfície, Descida no Fosso, Fundo do Mar, Oficina e Puzzles) foi desenhada com recurso ao padrão State. Graças a isto eliminou-se a necessidade de utilizar complexas e frágeis cadeias de if/else para validar ações. O motor do jogo sabe sempre em que estado está, rejeitando automaticamente comandos inválidos (por exemplo, tentar recolher minérios quando se está na superfície).

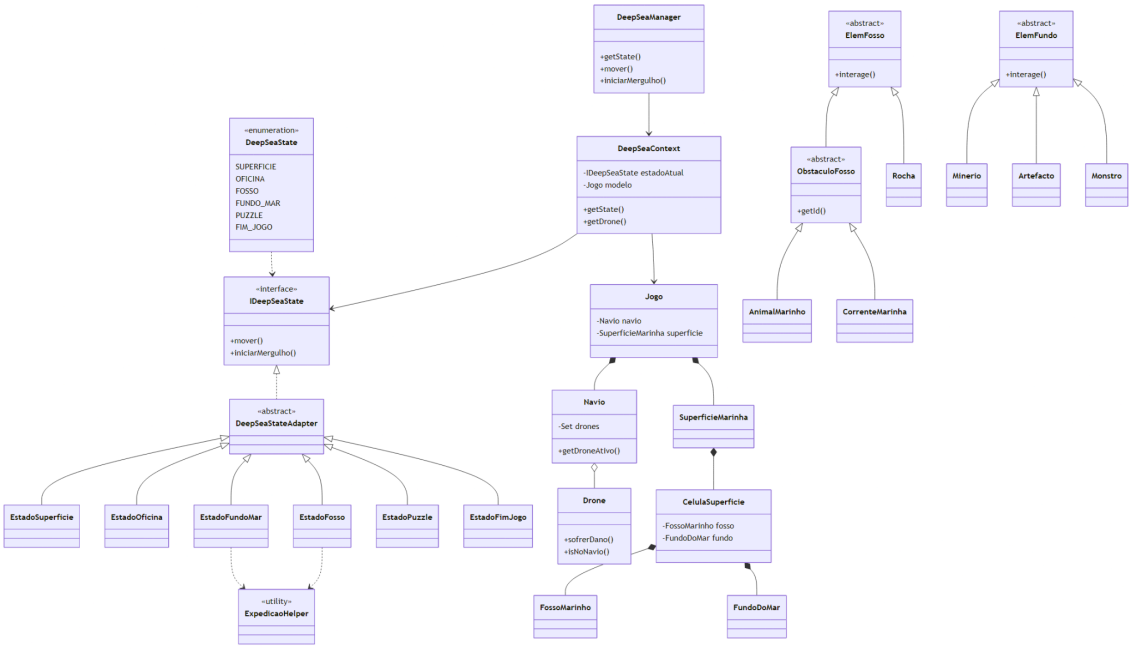
## 3. Diagrama de Estados




A FSM (Máquina de Estados Finitos) contém seis estados principais:

- **SUPERFÍCIE:** Ponto de partida. Permite gerir o navio, aceder à oficina ou iniciar o mergulho selecionando a zona.
- **FOSSO:** Fase de transição. O drone desce a grelha enfrentando obstáculos procedimentais que afetam integridade e combustível.
- **FUNDO_MAR:** Fase de exploração. Inclui sistema de "fog-of-war", recolha de minérios, encontros com monstros e localização de artefactos.
- **PUZZLE:** Ativado ao encontrar um artefacto. Consiste num mini-jogo de restabelecimento de ligação com limite de movimentos.
- **OFICINA:** Permite o upgrade das capacidades do drone utilizando os minérios recolhidos.
- **FIM_JOGO:** Estado terminal alcançado por vitória (recolha de todos os artefactos) ou derrota (destruição do drone/falta de recursos).

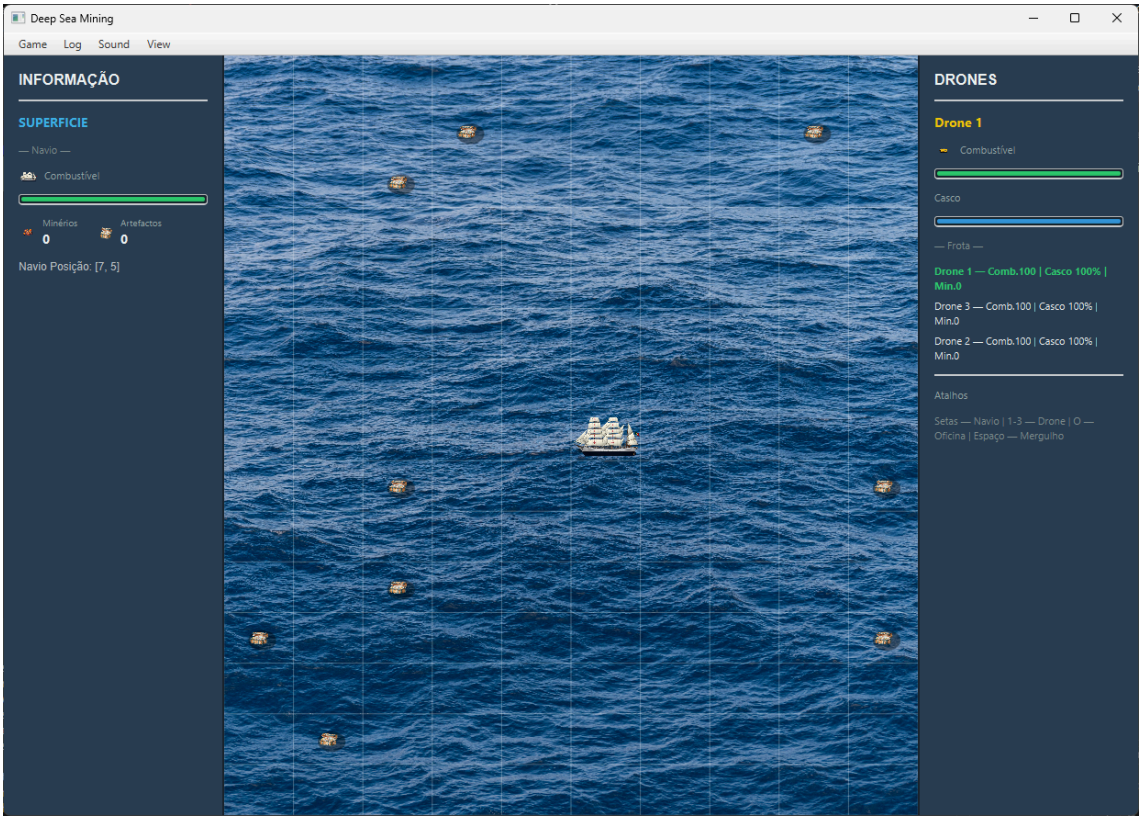
## 4. Diagrama de Classes e Estrutura



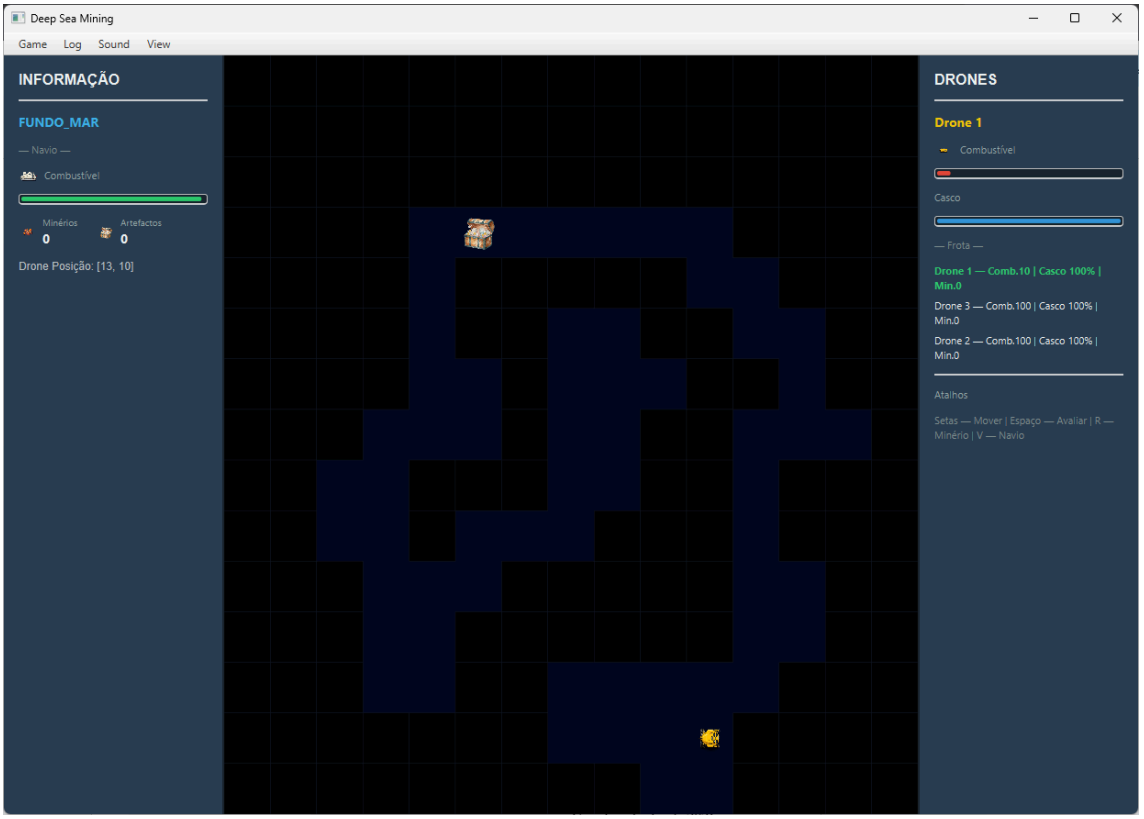
A estrutura de classes divide-se nos seguintes blocos operacionais:

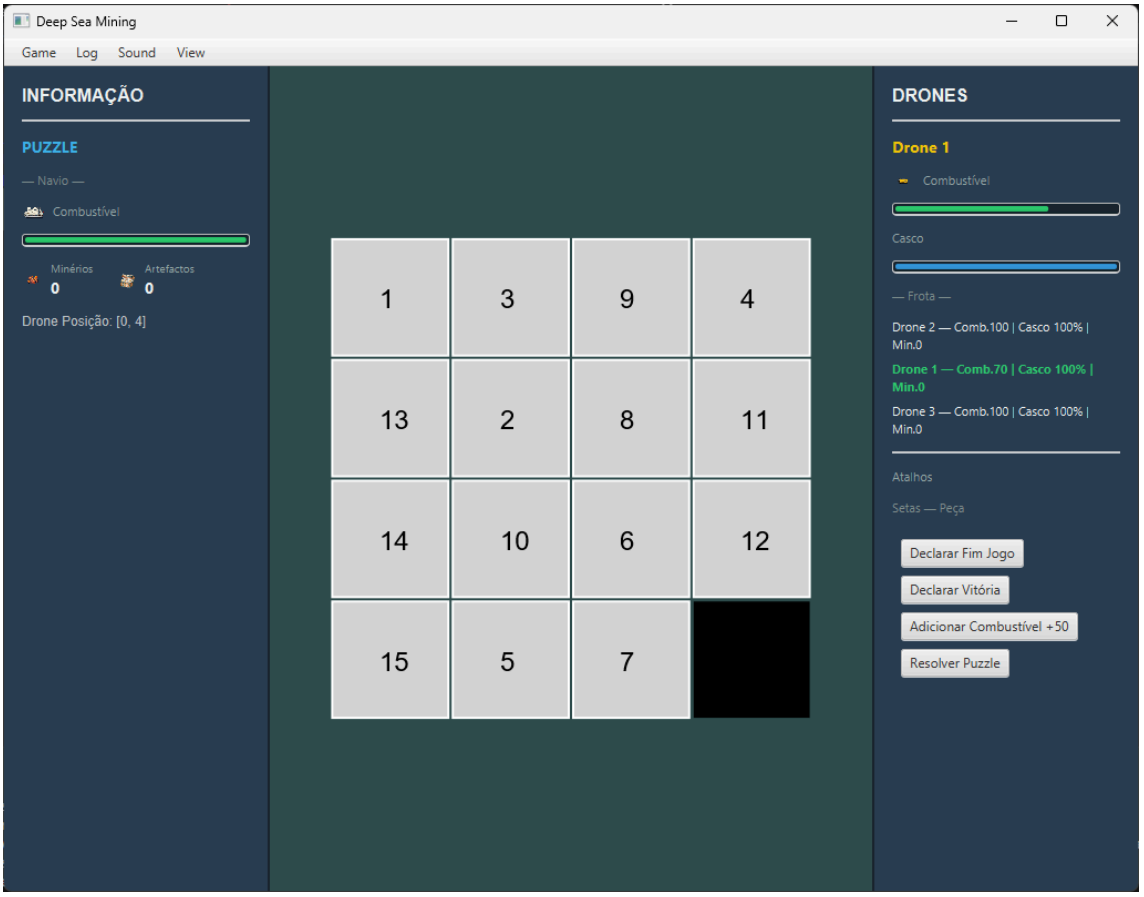
- Gestão e Estado: DeepSeaManager (Facade) e DeepSeaContext (Contexto da FSM).
- Modelo de Dados: Entidades principais como Navio, Drone e as representações espaciais SuperficieMarinha, FossoMarinho e FundoDoMar.
- Interface Gráfica: DeepSeaRoot gere a hierarquia visual. DeepSeaBoard e componentes associados (SuperficieView, DescidaView, FundoMarView) gerem o Canvas.
- Gestão de Recursos: SoundManager e ImageManager implementam o padrão Singleton para carregar e fornecer ficheiros multimédia on-demand sem duplicação em memória.

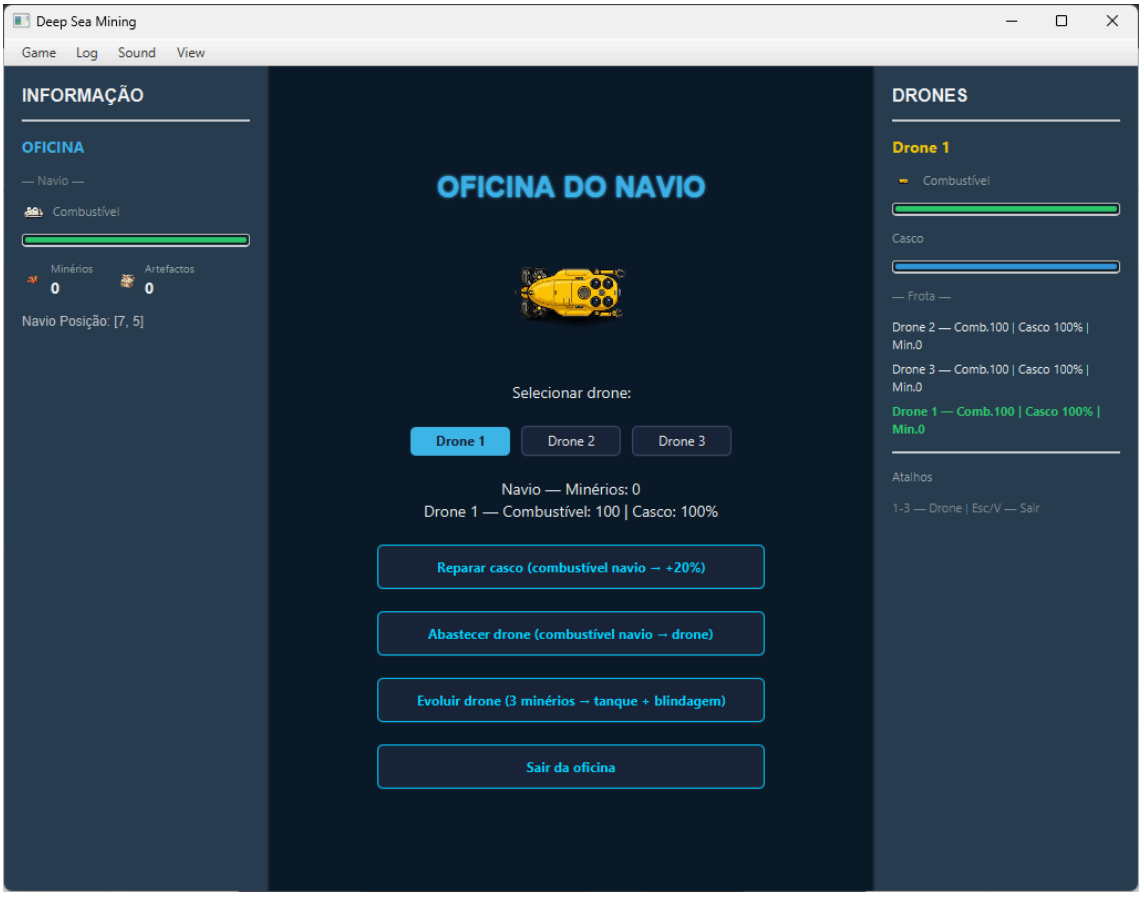
# 5. Explicação das Opções na UI

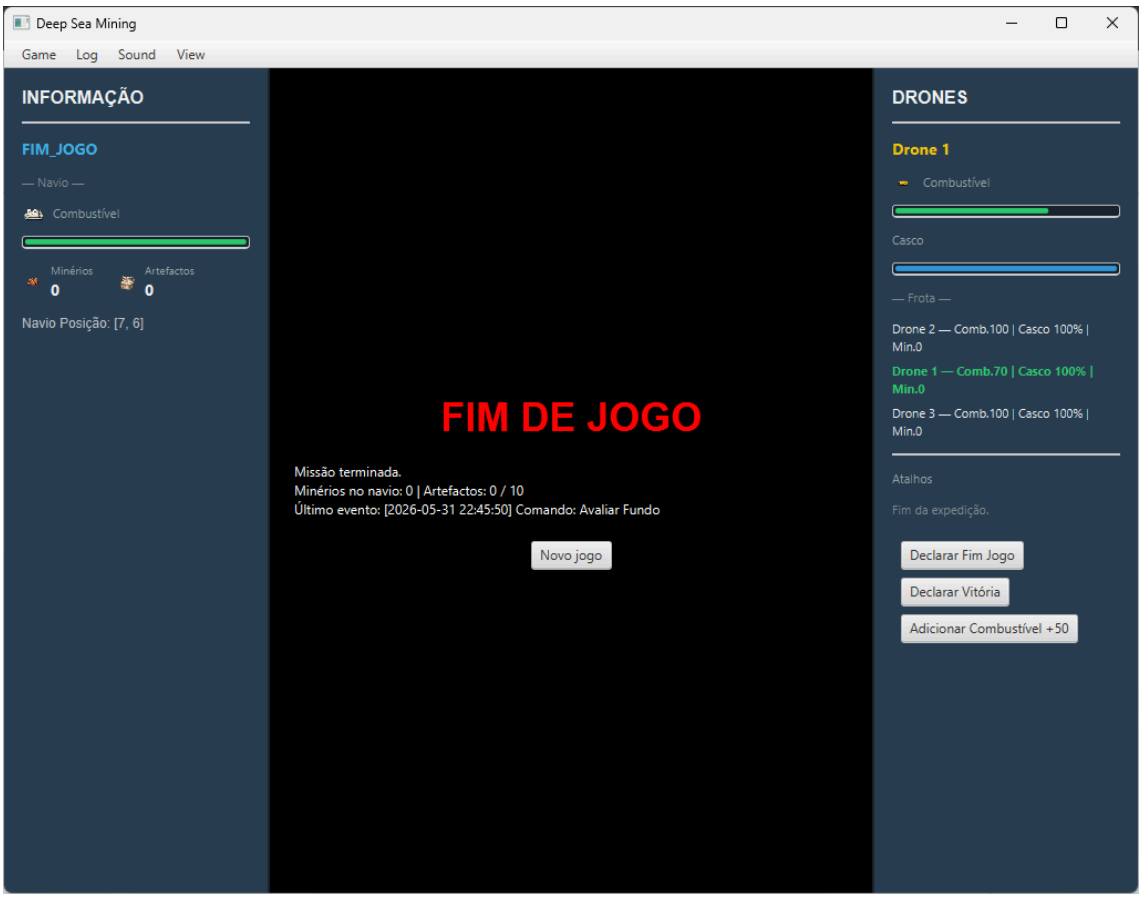



Deep Sea Mining
Game
Log
Sound
View
INFORMAÇÃO
FOSSO
— Navio —
Combustível
Minérios
0
Artefactos
0
Drone Posição: [0, 5]
DRONES
Drone 1
Combustível
Casco
— Frota —
Drone 1 — Comb.100 | Casco 100% | Min.0
Drone 3 — Comb.100 | Casco 100% | Min.0
Drone 2 — Comb.100 | Casco 100% | Min.0
Atalhos
Setas/Clique — Drone


Deep Sea Mining
Game
Log
Sound
View
INFORMAÇÃO
FUNDO_MAR
— Navio —
Combustível
Minérios
0
Artefactos
0
Drone Posição: [13, 10]
DRONES
Drone 1
Combustível
Casco
— Frota —
Drone 1 — Comb.10 | Casco 100% | Min.0
Drone 3 — Comb.100 | Casco 100% | Min.0
Drone 2 — Comb.100 | Casco 100% | Min.0
Atalhos
Setas — Mover | Espaço — Avaliar | R — Minério | V — Navio


Deep Sea Mining
Game
Log
Sound
View
INFORMAÇÃO
PUZZLE
— Navio —
Combustível
Minérios
0
Artefactos
0
Drone Posição: [0, 4]
1
3
9
4
13
2
8
11
14
10
6
12
15
5
7
DRONES
Drone 1
Combustível
Casco
— Frota —
Drone 2 — Comb.100 | Casco 100% | Min.0
Drone 1 — Comb.70 | Casco 100% | Min.0
Drone 3 — Comb.100 | Casco 100% | Min.0
Atalhos
Setas — Peça
Declarar Fim Jogo
Declarar Vitória
Adicionar Combustível +50
Resolver Puzzle


Deep Sea Mining
Game
Log
Sound
View
INFORMAÇÃO
OFICINA
— Navio —
Combustível
Minérios
0
Artefactos
0
Navio Posição: [7, 5]
OFICINA DO NAVIO
Selecionar drone:
Drone 1
Drone 2
Drone 3
Navio — Minérios: 0
Drone 1 — Combustível: 100 | Casco: 100%
Reparar casco (combustível navio → +20%)
Abastecer drone (combustível navio → drone)
Evoluir drone (3 minérios → tanque + blindagem)
Sair da oficina
DRONES
Drone 1
Combustível
Casco
— Frota —
Drone 2 — Comb.100 | Casco 100% | Min.0
Drone 3 — Comb.100 | Casco 100% | Min.0
Drone 1 — Comb.100 | Casco 100% | Min.0
Atalhos
1-3 — Drone | Esc/V — Sair


Deep Sea Mining
Game
Log
Sound
View
INFORMAÇÃO
FIM_JOGO
— Navio —
Combustível
Minérios
0
Artefactos
0
Navio Posição: [7, 6]
FIM DE JOGO
Missão terminada.
Minérios no navio: 0 | Artefactos: 0 / 10
Último evento: [2026-05-31 22:45:50] Comando: Avaliar Fundo
Novo jogo
DRONES
Drone 1
Combustível
Casco
— Frota —
Drone 2 — Comb.100 | Casco 100% | Min.0
Drone 1 — Comb.70 | Casco 100% | Min.0
Drone 3 — Comb.100 | Casco 100% | Min.0
Atalhos
Fim da expedição.
Declarar Fim Jogo
Declarar Vitória
Adicionar Combustível +50

A interface gráfica foi desenvolvida em JavaFX, respeitando rigorosamente a arquitetura MVC e utilizando o padrão Observer (PropertyChangeSupport) para a sincronização visual. A UI regista *listeners* genéricos (PROP_ESTADO, PROP_DADOS, PROP_LOG) que reagem automaticamente às mutações no modelo. Isto elimina a necessidade de *polling* e permite instanciar múltiplas janelas simultaneamente (como uma segunda janela de visualização) que partilham a mesma fonte de verdade sem falhas de sincronismo.

A nível de renderização e UX/UI, a equipa implementou as seguintes soluções para ultrapassar desafios técnicos específicos:

- **Cálculo Dinâmico das Grelhas (GridLayoutHelper):** Optou-se pela utilização do componente Canvas para garantir um desenho rápido e direto das matrizes de jogo. Para o redimensionamento dinâmico, foi criada a classe GridLayoutHelper, que calcula matematicamente o tamanho exato da célula — garantindo que se mantém rigorosamente quadrada — e ajusta os *offsets* (X e Y) para manter o conteúdo sempre centrado sempre que as dimensões da janela são alteradas.

- **Resolução do Ciclo de Vida do Canvas:** O redimensionamento livre da janela trouxe o desafio técnico de o JavaFX limpar automaticamente os desenhos do Canvas em tempo real. A solução de engenharia implementada passou por fazer o *override* do método nativo layoutChildren(), forçando o redesenho seguro apenas após a computação final do layout pelo motor gráfico.

- **Centralização de Interações e Usabilidade:** Para tornar a jogabilidade fluida, unificaram-se controlos. A título de exemplo, a barra de espaços aciona mecânicas distintas consoante o contexto (mergulhar o drone na Superfície ou ligar o radar no Fundo do Mar). Graças ao padrão State, a vista não necessita de processar condicionais complexas para evitar conflitos de *inputs*.

- **HUD Dinâmico e Estilização Visual:** O painel lateral (HUD) foi otimizado para reduzir a carga cognitiva, substituindo o excesso de texto por componentes gráficos dinâmicos como a ProgressBar. O limite visual da barra de combustível processa a percentagem lendo diretamente as configurações estáticas do domínio, sem valores literais acoplados. O *feedback* visual é complementado pela manipulação de propriedades CSS em tempo de execução (como o realce luminoso do drone ativo na frota).

**Gestão de Áudio Assíncrona (SoundManager):** A sonoplastia foi centralizada através do padrão Singleton. Para resolver os engasgos do motor de som originados por movimentos repetitivos e rápidos por parte do utilizador, desenvolveu-se uma lógica de controlo assíncrono que avalia os estados prévios de preparação do MediaPlayer (como READY ou PLAYING), garantindo uma transição fluida das faixas de áudio sem cortes abruptos.

## 6. Tabela de Tarefas

Implementação da FSM e transições de estado: Realizado

Mecânicas de gestão de recursos (combustível, integridade): Realizado

Geração procedimental do Fosso e Fundo do Mar: Realizado

Lógica de exploração (Fog-of-war) e sistema de combate: Realizado

Mecânica do mini-jogo (Puzzle): Realizado

Mecânica de upgrades (Oficina): Realizado

Interface gráfica JavaFX (Canvas e HUD): Realizado

Redimensionamento proporcional das grelhas: Realizado

Suporte para múltiplas janelas sincronizadas: Realizado

Implementação de efeitos sonoros e música (SoundManager): Realizado

Gravação/Carregamento do estado do jogo (Serialização): Realizado

Exportação e visualização de Logs: Realizado

Testes Unitários e Javadoc: Realizado